



CENG 408

**Innovative System Design and
Development 2**

2025-2026 Spring

Project:

**Syntagma: AI Powered Spaced Repetition System Based
Language Learning App**

Implementation Details

Team Members:

202211403 İlker Feza Çoban

202111027 İlayda Sümer

202111007 Elif Su Tufan

202211077 Alp Yılmaz

202211032 Sultan Gürbüz

202111417 Ömerhan Yiğit

Advisor: Abdul Kadir Görür

Table of Contents

1. INTRODUCTION	4
2. BACKEND APPLICATION	5
2.1 Technology Stack.....	5
2.2 Architectural Structure.....	5
2.3 Core Business Modules.....	5
2.3.1 Advanced Spaced Repetition (FSRS Algorithm).....	5
2.3.2 Flashcards & Collections.....	5
2.3.3 Sync and Offline Support.....	6
2.3.4 AI-Assisted Learning	6
2.3.5 Media and Ebook Management.....	6
2.3.6 Word Knowledge Tracking	6
2.4 Security & Exception Handling	6
2.5 Configuration Highlights.....	6
3. BROWSER EXTENSION	7
3.1 Purpose and Scope	7
3.2 System Context	7
3.3 Technology Stack.....	7
3.4 Runtime Architecture	8
3.4.1 Background Layer	8
3.4.2 Shared Layer.....	8
3.4.3 Content Layer	8
3.4.4 Extension Pages Layer	8
3.5 Core Data Model.....	9
3.5.1 User settings	9
3.5.2 Lexeme state.....	9
3.5.3 Flashcards	9
3.5.4 Dictionary store	9
3.5.5 Auxiliary storage keys.....	9
3.6 Message Contract and Communication.....	10
3.7 Page Parsing and Overlay Pipeline	10
3.7.1 Parsing flow.....	10
3.7.2 Overlay injection	10
3.7.3 User interaction logic	10
3.8 Word Popup and Learning Actions.....	11
3.9 Video Subsystem.....	11

3.9.1 Mounting strategy.....	11
3.9.2 Subtitle capture sources	11
3.9.3 Synchronization model.....	11
3.9.4 Interaction capabilities.....	11
3.9.5 Built-in Video Player Page	12
3.9.6 Audio Capture for Flashcards	12
3.10 Integrations	12
3.10.1 Dictionary audio API	12
3.10.2 Backend sync	12
3.10.3 AI Example Sentence Generation	12
3.11 Storage and Persistence Summary	12
3.12 Testing and Quality Status.....	13
3.13 Operational Notes and Known Limitations.....	13
3.14 Extension Pages	13
3.14.1 Workspace (card-creator.html).....	13
3.14.2 Auth Page (auth.html).....	13
3.14.3 Settings Page (options.html).....	13
3.14.4 eBook Reader Page (reader.html)	13
3.14.5 Popup Page (popup.html)	13
3.15 Internationalization (i18n) System	13
3.16 Contraction and Morphological Handling	13
3.17 CEFR Level Support.....	14
4. MOBILE APPLICATION.....	14
4.1 Purpose and Scope	14
4.2 Architecture	14
4.3 Data Model.....	14
4.4 Synchronization and Offline Strategy	15
4.5 Security	15
4.6 Testing and Quality.....	15
5. CONCLUSION	15
References	16

1. Introduction

Syntagma is an AI-powered, multi-platform language learning application designed to optimize vocabulary acquisition through advanced Spaced Repetition System (SRS) principles. The primary goal of the project is to lower cognitive load and facilitate long-term memory retention for Turkish-speaking users learning English. By integrating learning directly into the user's daily digital consumption—such as reading web articles or watching videos on platforms like YouTube—Syntagma provides a seamless, context-rich educational experience.

The Syntagma ecosystem is composed of three primary pillars:

- **Backend Application:** The robust core built on Spring Boot and PostgreSQL, responsible for managing user data, securing authentication via JWT, and processing the sophisticated FSRS (Free Spaced Repetition Scheduler) learning algorithm.
- **Browser Extension:** A Chrome Manifest V3 client that acts as the primary data capture tool. It features real-time text tokenization, contextual word popups, and advanced video subtitle interception to generate flashcards organically while the user browses the web.
- **Mobile Application:** A cross-platform app developed within the React Native/Expo ecosystem, allowing users to review their synced flashcards on the go, utilizing visual narratives and audio support to reinforce memory.

This document details the architectural decisions, technology stacks, core data models, and specific implementation workflows for each of these three subsystems, outlining how they function individually to support the overarching educational platform.

2. Backend Application

This section provides an overview of the backend system structure and technologies used in the project.

2.1 Technology Stack

The backend system is implemented using modern and scalable technologies:

- Framework: Spring Boot [1]
- Database: PostgreSQL [2]
- Security: Spring JWT [3]
- API Documentation: Swagger [4] / Postman [5]
- Build Tool: Gradle [6]
- Hibernate [7]

2.2 Architectural Structure

The backend follows a standard multi-layered architecture:

- Controllers (/controller): Exposes REST API endpoints. Controllers use DTOs for requests and responses.
- Services (/service): Encapsulates all business logic, algorithms, and orchestration.
- Repositories (/repository): Spring Data JPA interfaces interacting directly with the PostgreSQL database.
- Entities (/entity): JPA mapped domain classes defining the database schemas.
- DTOs (/dto): Strict request/response models to prevent entity leakage and simplify communication payloads.
- Security (/security): Stateless JWT filters enforcing authentication contexts on secured endpoints.
- Configuration (/config): Centralized configuration for security, OpenAPI, AI clients, and storage.

2.3 Core Business Modules

2.3.1 Advanced Spaced Repetition (FSRS Algorithm)

System uses Java implementation of the FSRS Algorithm, the modern standard recently adopted by Anki [8].

- FsrAlgorithm.java: Implements standard FSRS math, computing Difficulty, Stability, and Retrievability. 90% threshold rate. [9]
- SrsState & ReviewLog: Tracking individual learning states per flashcard.

2.3.2 Flashcards & Collections

- Hierarchical data structures using Collection, CollectionItem, and Flashcard.

- Includes MediaAsset references to bind images and audio logically to learning content without bloating the central database schemas.

2.3.3 Sync and Offline Support

- SyncEvent-based push/pull sync for client devices.
- Note: Entity-level sync processing is currently a placeholder for future expansion.

2.3.4 AI-Assisted Learning

- AI services for word explanations, sentence explanations, and translations.
- Requests are routed through a configurable external AI API provider.

2.3.5 Media and Ebook Management

- MediaAsset and StorageService provide S3-compatible presigned upload and download URLs.
- Ebook ingestion and progress tracking support reading-based learning workflows.

2.3.6 Word Knowledge Tracking

- WordKnowledge stores per-user vocabulary knowledge and related stats.

2.4 Security & Exception Handling

- JWT: Auth endpoints distribute JWT tokens; JwtAuthenticationFilter validates and bridges the user identity to secured resources.
- GlobalExceptionHandler: Provides uniform JSON-formatted error responses for EntityNotFound, validation failures, duplicates, and generic uncaught exceptions.

2.5 Configuration Highlights

- application.yml defines database, JWT, storage, logging, and AI settings.
- OpenAPI configuration documents secured endpoints with bearer authentication.
- Framework: Spring Boot [1]
- Database: PostgreSQL [2]
- Security: Spring JWT [3]
- API Documentation: Swagger [4] / Postman [5]
- Build Tool: Gradle [6]
- Hibernate [7]
- SrsState & ReviewLog: Tracking individual learning states per flashcard.

3. Browser Extension

3.1 Purpose and Scope

This section defines implementation details of the Syntagma browser extension under the extension/ directory.

Covered areas:

- Runtime architecture and component boundaries
- Build and packaging pipeline
- Core data and storage contracts
- Messaging and event-driven communication
- Content parsing and overlay logic
- Subtitle capture and video interaction subsystem
- Flashcard, dictionary, AI, and backend integration points

Test status and operational constraints

3.2 System Context

The extension is a Chrome Manifest V3 client that supports English learning for Turkish-speaking users by:

- tokenizing and lemmatizing page text
- applying word-level knowledge overlays
- showing contextual word popups
- capturing and processing subtitles
- creating flashcards and syncing learning state

Execution contexts:

1. Background Service Worker
2. Content Script Runtime
3. Extension Pages (Card Creator / Options / Auth / Reader / Video Player)

3.3 Technology Stack

- **Language:** TypeScript [10]
- **UI:** React 19, React DOM 19 [11]
- **Build:** Vite 8, Rollup outputs [12]
- **Extension Standard:** Chrome Manifest V3 [13]
- **Testing:** Vitest [14]
- **NLP support:** compromise [15]
- eBook reader: epubjs
- **Utilities:** uuid [16], lz-string [17], react-markdown [18]

3.4 Runtime Architecture

3.4.1 Background Layer

Responsibilities:

- process runtime message commands
- persist settings and lexical state
- orchestrate AI calls and stream chunks to tabs
- lookup translations from IndexedDB dictionary
- sync word status and flashcards to backend API
- create and handle extension context menu
- open options, card-creator, auth, reader, and video-player windows
- tab screenshot capture and tabCapture stream IDs for media

Notable implementation behaviour:

- dictionary population starts on worker startup
- keep-alive interval using `chrome.runtime.getPlatformInfo`
- message sender ID verification against own extension ID
- setting/status broadcasts to all tabs
- JWT refresh via X-Refreshed-Token response header
- cached signed URLs for backend flashcard media assets

3.4.2 Shared Layer

Responsibilities:

- central type definitions
- typed message envelopes
- storage helpers with defaults and user-scoped keys
- frequency table initialization and lookups
- lemmatization and irregular normalization cache
- backend AI client (`backend-ai.ts`)
- CEFR word lists and intake helpers
- i18n dictionary (`en/tr`)

3.4.3 Content Layer

Responsibilities:

- parse lifecycle orchestration
- token extraction and overlay rendering
- toolbar UI and page analysis
- popup-based word actions
- video subtitle overlay and transcript interactions
- Authentication gate: On initialization, the content script returns immediately if enabled is false or authToken is absent. Parsing, header bar, overlays, and video mode require a signed-in user.
- Video activation: Video mode initializes on YouTube watch and Netflix watch URLs after settings and lexemes load.

3.4.4 Extension Pages Layer

Extension pages are React applications opened as Chrome extension windows or tabs:

- card-creator.html — primary workspace (toolbar icon opens this; not default_popup)
- options.html — settings (manifest options_ui)
- auth.html — login and registration
- reader.html — EPUB eBook reader
- video-player.html — standalone video learning surface
- popup.html — built but not registered as toolbar popup in manifest.json

3.5 Core Data Model

3.5.1 User settings

Stored at chrome.storage.local.userSettings (merged with DEFAULT_SETTINGS).

Includes:

- Global: enabled, learnerLevel, supportLocale, targetLanguage, uiLocale (en | tr)
- Page parsing: showComprehensionHeader, showInlineTranslations, showLearningStatusColors, hideRareWords, autoParseOnLoad
- Reader: readerEnableInlineTranslations, readerShowLearningStatusColors, readerAutoParseChapterOnOpen, readerDefaultFontSize, readerLineHeight, readerTheme
- Video: autoPauseMode, subtitleDualMode, obscure modes, subtitle sizes, overlay opacity, pause/resume, scene skip, reveal options, offsets, sidebar
- AI / integrations: aiModel, aiApiKey (reserved), apiBaseUrl, authToken, authEmail, authUserId, activeCollectionId, activeCollectionName

3.5.2 Lexeme state

Stored under user-scoped key lexemes_{authUserId} when authUserId is set, otherwise lexemes.

Schema key: normalized lemma (string)

Schema value: LexemeEntry — status (unknown | learning | known | ignored), surface, type (word | phrase), trMeaning, audioUrl, cefr, frequency metadata, timestamps

Updates persisted locally and synced to /api/word-knowledge when authenticated.

3.5.3 Flashcards

Stored under flashcards_{authUserId} (or flashcards) as FlashcardPayload[].

Fields: id, lemma, surfaceForm, sentence, exampleSentence, sourceUrl, sourceTitle, trMeaning, knowledgeStatus, audioUrl, screenshotDataUrl, sentenceAudioDataUrl, videoTimestamp, deckName, tags, collectionId(s)

Created from word popup, content quick-add, video overlay, reader, and card-creator; synced with backend collections and media.

3.5.4 Dictionary store

IndexedDB: SyntagmaDictionaryDB, object store translations

Populated at service-worker startup from dictionary.json (web-accessible resource)

Lookup via LOOKUP_DICTIONARY message → dictionary-db.ts

dictionary.json must be placed at extension/dictionary.json before build (not committed to repo)

3.5.5 Auxiliary storage keys

flashcardMediaUrls — cached signed backend media URLs

flashcardScreenshotUrls — legacy screenshot cache

cardCreatorDraft:* — edit-session draft cards

comprehensionStats — per-page comprehension snapshots

3.6 Message Contract and Communication

Message model is command-response with async handling.

Extension command groups include:

- settings get/set
- single and bulk word status updates; upsert/delete word knowledge
- dictionary and audio lookup; word frequency lookup
- AI requests (explain word, explain sentence, translate, generate example)
- flashcard create/update/fetch/delete; collections fetch/create
- options/card creator/auth/reader/video-player window actions
- LOGIN, REGISTER, LOGOUT
- CAPTURE_TAB_SCREENSHOT, GET_TAB_CAPTURE_STREAM_ID
- OPEN_EXTERNAL_DICTIONARY (Tureng, Cambridge, Oxford, Merriam-Webster)

Background event groups include:

- AI stream chunk/done/error
- status changed notifications
- settings updated notifications

3.7 Page Parsing and Overlay Pipeline

3.7.1 Parsing flow

- TreeWalker traversal over document text nodes
- skipped tag and hidden element filtering
- regex tokenization for English-like tokens
- lemmatization and frequency enrichment
- contraction expansion and composite status resolution
- status mapping from lexeme state
- chunked processing with requestIdleCallback fallback

3.7.2 Overlay injection

- sentence wrappers: span.syn-sentence
- word wrappers: span[data-syn="lemma"]
- status class assignment (unknown/learning/known/ignored)
- optional inline TR translation tags
- scoped style injection and sentence-hover highlight behavior

3.7.3 User interaction logic

- click word to open popup
- shift-mode to pass clicks through overlays
- keyboard shortcuts (Alt+X)
- SPA navigation handling for YouTube and Netflix

3.8 Word Popup and Learning Actions

Word popup supports:

- status updates
- dictionary translation display
- frequency badge
- word audio playback (Dictionary API URL or speech synthesis fallback)
- external dictionary links

AI commands:

- explain word in sentence context
- explain sentence grammar
- translate sentence
- quick flashcard creation
- Quick flashcard creation may include `screenshotDataUrl` and `sentenceAudioDataUrl` from video context; card creator opens via `OPEN_CARD_CREATOR` with URL parameters or a draft storage key in edit mode.

Streaming behaviour:

- background dispatches chunked responses by `requestId`
- popup aggregates until done/error event

3.9 Video Subsystem

3.9.1 Mounting strategy

Video overlay mounts on `document.body` using fixed-position host to avoid player-overlay pointer-event conflicts.

3.9.2 Subtitle capture sources

YouTube sequence:

- timedtext API formats
- InnerTube/player-response extraction
- textTracks fallback

Netflix sequence:

- page-context interception of fetch/XHR
- manifest parsing for subtitle chunk URLs
- TTML chunk aggregation
- unified cue dispatch + DOM cache recovery path

3.9.3 Synchronization model

- YouTube auto-offset detection for subtitle alignment
- native subtitle layers hidden when extension subtitles are active

3.9.4 Interaction capabilities

- clickable subtitle words with popup integration
- multiple auto-pause modes
- silent-gap scene-skip strategies
- subtitle import (SRT/VTT)

- Netflix-safe seek via injected internal API

3.9.5 Built-in Video Player Page

- File: video-player.html → VideoPlayerApp.tsx. Opened via OPEN_VIDEO_PLAYER.
- Dedicated HTML5 video surface independent of YouTube/Netflix DOM
- Import SRT/VTT subtitles; tokenized clickable words
- Shared auto-pause, reveal, and popup flows with in-page video mode
- Flashcard creation with video-player source metadata

3.9.6 Audio Capture for Flashcards

In-page video (audio-recorder.ts):

- Uses HTMLVideoElement.captureStream() and MediaRecorder (no tabCapture for cue clips)
- Records audio aligned to subtitle cue times (max 30 s per cue)
- Produces WebM data URLs as sentenceAudioDataUrl on flashcards

Background service worker:

- tabCapture permission for GET_TAB_CAPTURE_STREAM_ID
- tabs.captureVisibleTab for CAPTURE_TAB_SCREENSHOT
- Backend MediaAsset signed URLs cached in flashcardMediaUrls

3.10 Integrations

3.10.1 Dictionary audio API

Default pronunciation source: <https://api.dictionaryapi.dev/api/v2/entries/en/{word}>

3.10.2 Backend sync

Default base fallback: <https://syntagma.omerhanyigit.online>

Used for:

- word knowledge sync (single, batch, and level intake)
- flashcard create/list/update/delete and collections
- AI explain/translate/generate-example endpoints
- media asset URLs for flashcard screenshot and audio
- Token refresh: X-Refreshed-Token header updates authToken in userSettings automatically.

3.10.3 AI Example Sentence Generation

Message: GENERATE_EXAMPLE_SENTENCE

Endpoint: POST /api/ai/generate-example (requires authToken)

Used from card creator to populate exampleSentence on FlashcardPayload

Single request-response (not streamed)

3.11 Storage and Persistence Summary

chrome.storage.local:

- userSettings — global UserSettings
- lexemes or lexemes_{userId} — Record<lemma, LexemeEntry>
- flashcards or flashcards_{userId} — FlashcardPayload[]
- comprehensionStats — page comprehension snapshots

- flashcardMediaUrls — cached backend media URLs
- cardCreatorDraft:* — ephemeral edit drafts
- User scoping via userScopedKey(base, authUserId) isolates data per account on the same browser profile.
- IndexedDB: SyntagmaDictionaryDB/translations — populated from dictionary.json.

3.12 Testing and Quality Status

See Section 3.18 for the current automated test inventory and manual QA prerequisites.

3.13 Operational Notes and Known Limitations

- dictionary.json is not in the repository; add extension/dictionary.json before npm run build.
- Content-script features require authToken (sign in via auth.html or card creator).
- Toolbar icon opens card-creator.html, not popup.html.
- Netflix capture depends on document_start interceptor; site changes may require updates.
- Production AI uses backend endpoints; client OpenRouter (ai.ts) is unused.
- forvoApiKey exists in settings only; pronunciation uses Dictionary API and speech synthesis.
- index.html / App.tsx is a dev scaffold, not a shipped extension entry.

3.14 Extension Pages

3.14.1 Workspace (card-creator.html)

CardCreatorApp.tsx — central hub: home panel, flashcard create/edit, word browser, flashcard list, inline auth, collection picker, AI example sentences. Opened by toolbar click, context menu, and OPEN_CARD_CREATOR.

3.14.2 Auth Page (auth.html)

AuthApp.tsx — login/register, password rules, learner level on registration, persists JWT via LOGIN/REGISTER messages. Can run standalone or inline in card creator.

3.14.3 Settings Page (options.html)

OptionsApp.tsx — manifest options_ui: parsing, video, reader, AI URL, account, word browser, flashcard management.

3.14.4 eBook Reader Page (reader.html)

ReaderApp.tsx — epubjs EPUB reader, chapter overlays, whole-book analysis (reader-analysis.ts), comprehension stats, word popup and flashcard actions, light/sepia/dark themes.

3.14.5 Popup Page (popup.html)

PopupPageApp.tsx — compact enable toggle; built but not bound as default_popup in manifest.

3.15 Internationalization (i18n) System

Module: src/shared/i18n.tsx

Locales: English (en), Turkish (tr); persisted as UserSettings.uiLocale

t(key, locale) and useT hook; LocaleToggle component

Used in card creator, options, auth, reader, and feature description strings

3.16 Contraction and Morphological Handling

Contractions (parser.ts, video/tokenizer.ts):

- CONTRACTIONS map surfaces like don't to constituent lemmas [do, not]
- resolveContractionStatus: unknown if any part unknown; else learning if any learning; else known
- Lemmatization (lemmatizer.ts):
- compromise for verb infinitive and noun singular forms
- in-memory cache; large IRREGULAR_MAP pre-seeded at module load
- synchronous API required for content-script IIFE bundle
- Multi-word phrases matched from assets/phrases-en.json as type phrase.

3.17 CEFR Level Support

LearnerLevel maps to CEFR: beginner→A1, elementary→A2, intermediate→B1, upper-intermediate→B2, pre-advanced→C1, advanced→C2

Word lists: src/assets/levels/a1.txt ... c2.txt and cefr-words-en.json

cefr-intake.ts: POST /api/word-knowledge/level on level change; syncs local lexemes from backend KNOWN set

pre-advanced selects C1 corpus for intake between B2 and C2.

4. MOBILE APPLICATION

4.1 Purpose and Scope

Syntagma has a mobile application developed within the Expo ecosystem that optimizes the language learning process based on Spaced Repetition System (SRS) principles. The application aims to facilitate the transfer of vocabulary into long-term memory through its academic-focused design and machine-learning-supported algorithms. Focusing initially on the English-Turkish language pair, the project offers a sustainable study discipline by reducing cognitive load through visual narratives and gamification elements.

4.2 Architecture

The application is built on React 19 and React Native [19] 0.81 technologies within the Expo [20] ecosystem, offering cross-platform support (iOS/Android).

- Modular Structure: Navigation management is structured in a hierarchical model (Authentication Stack, Bottom Tab Navigator, and Operational Stack) utilizing the React Navigation library.
- Interface Technologies: Expo-Linear-Gradient is used for visual continuity, and Expo-Font is utilized for customized typography (DM Sans and Playfair Display). Consistent experience across all devices is ensured via hardware-level "Safe Area" management.
- Algorithm Layer: In future phases, the machine learning-based FSRS (Free Spaced Repetition Scheduler) algorithm, which analyses the user's memory state, will be fully implemented.

4.3 Data Model

Syntagma's data model is structured to track the learning process:

- User Data: Secure JWT-based authentication data.
- Content Structure: Flashcards consist of a front side (word, example sentence, phonetic info) and an expanded back side (translation, voice pronunciation, visual context).
- Statistical Data: Metrics such as daily review counts, learned word volume, and accuracy rates are fetched via API and visualized on a dashboard.

4.4 Synchronization and Offline Strategy

A multi-layered strategy is followed for an uninterrupted user experience:

- Offline Support: To maintain functionality in environments without internet access (e.g., airplanes, subways), data is stored in Local Storage.
- Cloud Synchronization: Local data is automatically synchronized with the backend once an internet connection is established.
- Cross-Platform Integration: Instant synchronization of cards created via a Chrome extension with the mobile app is planned for future implementation.

4.5 Security

Application security and data integrity are protected through the following methods:

- Authentication: All login and registration processes are managed with a secure JWT (JSON Web Token) infrastructure.
- Data Transmission: All data flow between the client and server is carried out over industry-standard encryption protocols.
- Access Control: Operational layers and user-specific statistics are restricted within the framework of authorized sessions.

4.6 Testing and Quality

The application undergoes design and functional testing processes to maintain quality standards:

- Visual Quality: The consistency of the academic design system, featuring #F2EDE4 (Beige) and #6B4226 (Brown) tones, is tested across different screen resolutions.
- Performance: A low-latency experience is targeted through hardware-level Safe Area management and resource usage optimization.
- User Feedback: The accuracy of the FSRS algorithm is analyzed through the "Remembered/Forgot" cycle, alongside the impact of the smart notification system (fewer than 3 per day) on the user.

5. CONCLUSION

This document has outlined the comprehensive technical foundation and implementation details of the Syntagma language learning platform. By breaking down the system into three distinct yet complementary applications, the Backend, the Browser Extension, and the Mobile Application, we

have established a scalable, modern architecture capable of supporting an advanced, AI-driven educational tool.

The Backend provides a secure, high-performance environment using Spring Boot and PostgreSQL to handle complex SRS computations, secure API routing, and data persistence.

The Browser Extension serves as a data-entry point, with custom video-overlay injections to turn everyday web browsing and media consumption into active-learning opportunities. Finally, the

Mobile Application ensures that the learning process remains continuous and accessible, utilizing a modular React Native architecture to deliver a smooth, cross-platform review experience.

References

- [1] S. Contributors, "Spring Boot Reference Documentation," Spring, [Online]. Available: <https://docs.spring.io/spring-boot/index.html>. [Accessed 10 April 2026].
- [2] The PostgreSQL Global Development Group, "PostgreSQL Documentation," PostgreSQL, [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed 10 April 2026].
- [3] S. Contributors, "Jwt (Spring Security API)," Spring, [Online]. Available: <https://docs.spring.io/spring-security/site/docs/current/api/org/springframework/security/oauth2/jwt/Jwt.html>. [Accessed 10 April 2026].
- [4] SmartBear Software, "OpenAPI Specification," Swagger, [Online]. Available: https://swagger.io/docs/specification/v3_0/about/. [Accessed 10 April 2026].
- [5] I. Postman, "Postman Introduction," Postman Learning Center, [Online]. Available: <https://learning.postman.com/docs/introduction/overview>. [Accessed 10 April 2026].
- [6] Gradle Inc., "Gradle User Manual," Gradle, [Online]. Available: <https://docs.gradle.org/current/userguide/userguide.html>. [Accessed 10 April 2026].
- [7] Red Hat, Inc., "Hibernate ORM User Guide," Hibernate, [Online]. Available: https://docs.hibernate.org/orm/current/userguide/html_single/. [Accessed 10 April 2026].
- [8] o.-s.-r. Contributors, "fsrs4anki: A modern Spaced Repetition algorithm," GitHub, [Online]. Available: <https://github.com/open-spaced-repetition/fsrs4anki>. [Accessed 10 April 2026].

- [9] open-spaced-repetition Contributors, “rs-fsrs-java: Java bindings for rs-fsrs,” GitHub, [Online]. Available: <https://github.com/open-spaced-repetition/rs-fsrs-java>. [Accessed 10 April 2026].
- [10] Microsoft, “TypeScript: JavaScript With Syntax For Types,” [Online]. Available: <https://www.typescriptlang.org/>. [Accessed 10 April 2026].
- [11] Meta Platforms, Inc., “React: The library for web and native user interfaces,” React, [Online]. Available: <https://react.dev/>. [Accessed 10 4 2026].
- [12] E. Y. a. V. Contributors., “Vite: Next Generation Frontend Tooling,” Vite, [Online]. Available: <https://vite.dev/>. [Accessed 10 April 2026].
- [13] Google, “Manifest V3,” Chrome Developers, [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/>. [Accessed 10 April 2026].
- [14] V. Contributors, “Vitest: A blazing fast unit test framework powered by Vite,” Vitest, [Online]. Available: <https://vitest.dev/>. [Accessed 10 April 2026].
- [15] S. Kelly, “compromise: modest natural-language processing,” GitHub, [Online]. Available: <https://github.com/spencermountain/compromise>. [Accessed 10 April 2026].
- [16] uuid Contributors, “uuid,” npm, [Online]. Available: <https://www.npmjs.com/package/uuid>. [Accessed 10 April 2026].
- [17] Pieroxy, “lz-string,” npm, [Online]. Available: <https://www.npmjs.com/package/lz-string>. [Accessed 10 4 2026].
- [18] r. Contributors, “react-markdown,” npm, [Online]. Available: <https://www.npmjs.com/package/react-markdown>. [Accessed 10 April 2026].
- [19] I. Meta Platforms, “React Native: A framework for building native apps using React,” React Native, [Online]. Available: <https://reactnative.dev/>. [Accessed 10 April 2026].
- [20] Expo, “Expo Documentation,” Expo, [Online]. Available: <https://docs.expo.dev/>. [Accessed 10 April 2026].